

EXPERIMENT 18

IMPLEMENTATION OF SHA-512 HASH FUNCTION

AIM

To implement the SHA-512 (Secure Hash Algorithm 512) cryptographic hash function in C and generate the 512-bit message digest for a given input message.

ALGORITHM

1. Accept the input message from the user.
2. Convert the message into a sequence of bytes.
3. Append a single 1 bit to the message.
4. Append 0 bits until the message length is 896 bits modulo 1024.
5. Append the original message length as a 128-bit value.
6. Divide the padded message into 1024-bit blocks.
7. Initialize eight 64-bit hash values:
 - H0
 - H1
 - H2
 - H3
 - H4
 - H5
 - H6
 - H7
8. For every 1024-bit block, create 80 message schedule words.
9. Perform 80 rounds of SHA-512 compression using:
 - Choice function
 - Majority function
 - $\Sigma 0$ and $\Sigma 1$
 - $\sigma 0$ and $\sigma 1$
10. Add the resulting working variables to the hash values.
11. Process all blocks.
12. Concatenate the eight 64-bit hash values to obtain the 512-bit SHA-512 digest.

PROGRAM:

```
#include <stdio.h>

#include <stdint.h>

#include <string.h>

#include <stdlib.h>
```

/* SHA-512 Constants */

```
static const uint64_t K[80] = {  
    0x428a2f98d728ae22ULL, 0x7137449123ef65cdULL,  
    0xb5c0fbcfec4d3b2fULL, 0xe9b5dba58189dbbcULL,  
    0x3956c25bf348b538ULL, 0x59f111f1b605d019ULL,  
    0x923f82a4af194f9bULL, 0xab1c5ed5da6d8118ULL,  
    0xd807aa98a3030242ULL, 0x12835b0145706fbeULL,  
    0x243185be4ee4b28cULL, 0x550c7dc3d5ffb4e2ULL,  
    0x72be5d74f27b896fULL, 0x80deb1fe3b1696b1ULL,  
    0x9bdc06a725c71235ULL, 0xc19bf174cf692694ULL,  
    0xe49b69c19ef14ad2ULL, 0xefbe4786384f25e3fULL,  
    0x0fc19dc68b8cd5b5ULL, 0x240ca1cc77ac9c65ULL,  
    0x2de92c6f592b0275ULL, 0x4a7484aa6ea6e483ULL,  
    0x5cb0a9dcbbd41fbd4ULL, 0x76f988da831153b5ULL,  
    0x983e5152ee66dfabULL, 0xa831c66d2db43210ULL,  
    0xb00327c898fb213fULL, 0xbf597fc7beef0ee4ULL,  
    0xc6e00bf33da88fc2ULL, 0xd5a79147930aa725ULL,  
    0x06ca6351e003826fULL, 0x142929670a0e6e70ULL,  
    0x27b70a8546d22ffcULL, 0x2e1b21385c26c926ULL,  
    0x4d2c6dfc5ac42aedULL, 0x53380d139d95b3dfULL,  
    0x650a73548baf63deULL, 0x766a0abb3c77b2a8ULL,  
    0x81c2c92e47edaee6ULL, 0x92722c851482353bULL,  
    0xa2bfe8a14cf10364ULL, 0xa81a664bbc423001ULL,  
    0xc24b8b70d0f89791ULL, 0xc76c51a30654be30ULL,  
    0xd192e819d6ef5218ULL, 0xd69906245565a910ULL,  
    0xf40e35855771202aULL, 0x106aa07032bbd1b8ULL,  
    0x19a4c116b8d2d0c8ULL, 0x1e376c085141ab53ULL,  
    0x2748774cdf8eeb99ULL, 0x34b0bcb5e19b48a8ULL,
```

```

0x391c0cb3c5c95a63ULL, 0x4ed8aa4ae3418acbULL,
0x5b9cca4f7763e373ULL, 0x682e6ff3d6b2b8a3ULL,
0x748f82ee5defb2fcULL, 0x78a5636f43172f60ULL,
0x84c87814a1f0ab72ULL, 0x8cc702081a6439ecULL,
0x90befffa23631e28ULL, 0xa4506cebbe82bde9ULL,
0xbef9a3f7b2c67915ULL, 0xc67178f2e372532bULL,
0xca273eceea26619cULL, 0xd186b8c721c0c207ULL,
0xead7dd6cde0eb1eULL, 0xf57d4f7fee6ed178ULL,
0x06f067aa72176fbaULL, 0x0a637dc5a2c898a6ULL,
0x113f9804bef90daeULL, 0x1b710b35131c471bULL,
0x28db77f523047d84ULL, 0x32caab7b40c72493ULL,
0x3c9ebe0a15c9bebcULL, 0x431d67c49c100d4cULL,
0x4cc5d4becb3e42b6ULL, 0x597f299cfc657e2aULL,
0x5fcb6fab3ad6faecULL, 0x6c44198c4a475817ULL
};

```

```

/* Right rotation */

```

```

uint64_t ROTR(uint64_t x, int n)
{
    return (x >> n) | (x << (64 - n));
}

```

```

/* SHA-512 functions */

```

```

uint64_t Ch(uint64_t x, uint64_t y, uint64_t z)
{
    return (x & y) ^ (~x & z);
}

```

```

uint64_t Maj(uint64_t x, uint64_t y, uint64_t z)

```

```
{  
    return (x & y) ^ (x & z) ^ (y & z);  
}
```

```
uint64_t BigSigma0(uint64_t x)  
{  
    return ROTR(x, 28) ^ ROTR(x, 34) ^ ROTR(x, 39);  
}
```

```
uint64_t BigSigma1(uint64_t x)  
{  
    return ROTR(x, 14) ^ ROTR(x, 18) ^ ROTR(x, 41);  
}
```

```
uint64_t SmallSigma0(uint64_t x)  
{  
    return ROTR(x, 1) ^ ROTR(x, 8) ^ (x >> 7);  
}
```

```
uint64_t SmallSigma1(uint64_t x)  
{  
    return ROTR(x, 19) ^ ROTR(x, 61) ^ (x >> 6);  
}
```

```
/* Read 64-bit big-endian word */  
uint64_t read64(const unsigned char *p)  
{  
    return ((uint64_t)p[0] << 56) |  
           ((uint64_t)p[1] << 48) |
```

```

        ((uint64_t)p[2] << 40) |
        ((uint64_t)p[3] << 32) |
        ((uint64_t)p[4] << 24) |
        ((uint64_t)p[5] << 16) |
        ((uint64_t)p[6] << 8) |
        ((uint64_t)p[7]);
    }

/* SHA-512 */

void sha512(const unsigned char *message, size_t length)
{
    uint64_t H[8] = {
        0x6a09e667f3bcc908ULL,
        0xbb67ae8584caa73bULL,
        0x3c6ef372fe94f82bULL,
        0xa54ff53a5f1d36f1ULL,
        0x510e527fade682d1ULL,
        0x9b05688c2b3e6c1fULL,
        0x1f83d9abfb41bd6bULL,
        0x5be0cd19137e2179ULL
    };

    /*
     * SHA-512 padding:
     * message + 0x80 + zeros + 128-bit length
     */
    size_t newLength = length + 1;

    while (newLength % 128 != 112)

```

```

    newLength++;

size_t totalLength = newLength + 16;

unsigned char *data =
    (unsigned char *)calloc(totalLength, 1);

if (data == NULL)
{
    printf("Memory allocation failed.\n");
    return;
}

memcpy(data, message, length);

/* Append 1 bit */
data[length] = 0x80;

/*
 * Append message length as a 128-bit
 * big-endian integer.
 *
 * For normal lab-sized input, the high
 * 64 bits are zero.
 */
uint64_t bitLength = (uint64_t)length * 8;

for (int i = 0; i < 8; i++)
{

```

```
data[newLength + 8 + i] =  
    (unsigned char)(bitLength >> (56 - 8 * i));  
}
```

```
/* Process each 1024-bit block */
```

```
for (size_t offset = 0;  
     offset < totalLength;  
     offset += 128)
```

```
{  
    uint64_t W[80];
```

```
/* First 16 words */
```

```
for (int i = 0; i < 16; i++)  
{  
    W[i] = read64(data + offset + i * 8);  
}
```

```
/* Remaining words */
```

```
for (int i = 16; i < 80; i++)  
{  
    W[i] =  
        SmallSigma1(W[i - 2])  
        + W[i - 7]  
        + SmallSigma0(W[i - 15])  
        + W[i - 16];  
}
```

```
uint64_t a = H[0];
```

```
uint64_t b = H[1];
```

```

uint64_t c = H[2];
uint64_t d = H[3];
uint64_t e = H[4];
uint64_t f = H[5];
uint64_t g = H[6];
uint64_t h = H[7];

/* 80 rounds */
for (int i = 0; i < 80; i++)
{
    uint64_t T1 =
        h + BigSigma1(e)
        + Ch(e, f, g)
        + K[i] + W[i];

    uint64_t T2 =
        BigSigma0(a)
        + Maj(a, b, c);

    h = g;
    g = f;
    f = e;
    e = d + T1;
    d = c;
    c = b;
    b = a;
    a = T1 + T2;
}

```



```

        H[0] += a;
        H[1] += b;
        H[2] += c;
        H[3] += d;
        H[4] += e;
        H[5] += f;
        H[6] += g;
        H[7] += h;
    }

    free(data);

    printf("\nSHA-512 HASH\n");
    printf("-----\n");

    for (int i = 0; i < 8; i++)
    {
        printf("%016llx",
            (unsigned long long)H[i]);
    }

    printf("\n");
}

int main()
{
    char message[1000];

    printf("SHA-512 HASH FUNCTION\n");

```

```

printf("-----\n");

printf("Enter message: ");

fgets(message, sizeof(message), stdin);

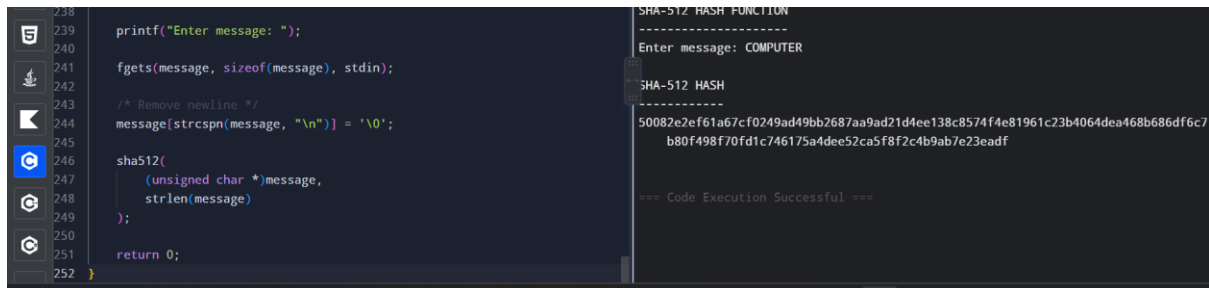
/* Remove newline */
message[strcspn(message, "\n")] = '\0';

sha512(
    (unsigned char *)message,
    strlen(message)
);

return 0;
}

```

OUTPUT:



The screenshot shows a code editor on the left and a terminal window on the right. The code in the editor is a C program that prompts the user to enter a message, reads it using fgets, removes the trailing newline character, and then calculates the SHA-512 hash using a custom sha512 function. The terminal output shows the program running, the user entering 'COMPUTER', and the resulting 512-bit hash displayed in hexadecimal format. The hash is: 50082e2ef61a67cf0249ad49bb2687aa9ad21d4ee138c8574f4e81961c23b4064dea468b686df6c7b80f498f70fd1c746175a4dee52ca5f8f2c4b9ab7e23eadf. The terminal also shows 'Code Execution Successful'.

```

238 printf("Enter message: ");
239
240 fgets(message, sizeof(message), stdin);
241
242 /* Remove newline */
243 message[strcspn(message, "\n")] = '\0';
244
245 sha512(
246     (unsigned char *)message,
247     strlen(message)
248 );
249
250 return 0;
251
252 }

```

```

SHA-512 HASH FUNCTION
-----
Enter message: COMPUTER

SHA-512 HASH
-----
50082e2ef61a67cf0249ad49bb2687aa9ad21d4ee138c8574f4e81961c23b4064dea468b686df6c7
b80f498f70fd1c746175a4dee52ca5f8f2c4b9ab7e23eadf

=== Code Execution Successful ===

```

RESULT

Thus, the SHA-512 hash function was successfully implemented in C, and a 512-bit message digest was generated for the given input message.